Assignment 1: Data Extraction & Integration

Course: INFOMDW (Data Wrangling) — Applied Data Science
Group: 8
Group members: Theodora Kyriakou, Satyam Kumar, Sultonbek Abdulloev, Sonja Wellman
Selected event: ATP Grand Slams + Masters 1000, season 2025

Table of contents

Table of contents

Scope

Task 1: Database Design

- 1.1 ER diagram
- 1.2 Cardinalities and how we represent them
- 1.3 Primary keys, foreign keys and referential integrity
  - Primary keys
  - Foreign keys and policies
- 1.4 SQL file
- How to run the database
- 1.5 BCNF check of the entity tables

Task 2: Querying the database

- Query 1: join
- Query 2: aggregate function
- Query 3: nested query
- Query 4: self join
- Query 5: set operation

Task 3: Data Extraction in Python

- 3.1 Connecting the database and extracting query results
- 3.2 Reading a relation into a Pandas dataframe
- 3.3 Hashing function and collision rate

The end

Scope

This database models the Grand Slam and ATP Masters 1000 tennis tournaments in the 2025 season. Our aim was to make the database as realistic, reproducible and easy to enrich as possible.

We designed this database with 13 different tables based on the distinction between information that belongs to a player, a tournament series, a particular edition, an event and an individual match, because all of these objects can recur independently. Each table has a specific purpose, and we thought carefully about where each information should live. One player can enter multiple events, one series has multiple editions, and one event contains many matches. There are also tables operating as lookups (like country, venue, round) or as junction tables (like coach_player). Storing each fact in its own table reduces duplication and prevents inconsistencies when updating or expanding our database. Junction tables represent many-to-many relationships and hold attributes of those relationships, such as a player's seed and ranking in a specific event. Foreign keys then connect the tables and ensure that match participants are registered for the correct event. This structure preserves the tournament data of the 2025 season while making it suitable for joins and analysis.

Although we populated the database with men's singles only, it was designed to accommodate women's singles too. Women's draws for the specific tournament categories can be added by inserting new records in the tables without changing anything on the schema. Same applies to adding new tournament or editions, a new tournament is a new series, and a new edition is a new year of an existing series (as long as its category is one of the ones that the schema allows).

Task 1: Database Design

1.1 ER diagram

We started thinking about the design while referencing to some publicly available CSV files containing data about the tennis tournaments ATP Grand Slams and Masters 1000 (source link: https://github.com/Tennismylife/TML-Database). At first, there were two tables containing data about the players and the matches retrieved from that public website. Also, because we thought that venues are an important part of the tennis tournaments, we created a CSV file (using AI) with data for tennis venues too. However, all three CSV files contained repeated information a lot. We used these tables as a starting point, we split their attributes into more consistent tables, and we added new attributes and tables (like country, coach) to make the database more complete and efficient. This resulted into 13 tables.

The way we ended up with these 13 tables was by using two rules.

- **Rule 1:** One table per real-world object/concept. So, each table and its attributes should describe exactly one thing.
- **Rule 2:** One relationship table per many-to-many combinations. There are some relationships between real-world objects that require many records from both sides (like a coach can have several players and a player can have several coaches). A simple foreign key in such cases would need storing values as lists which is not the most efficient way. So, each many-to-many combination with its attributes (describing the relationship) gets its own table.

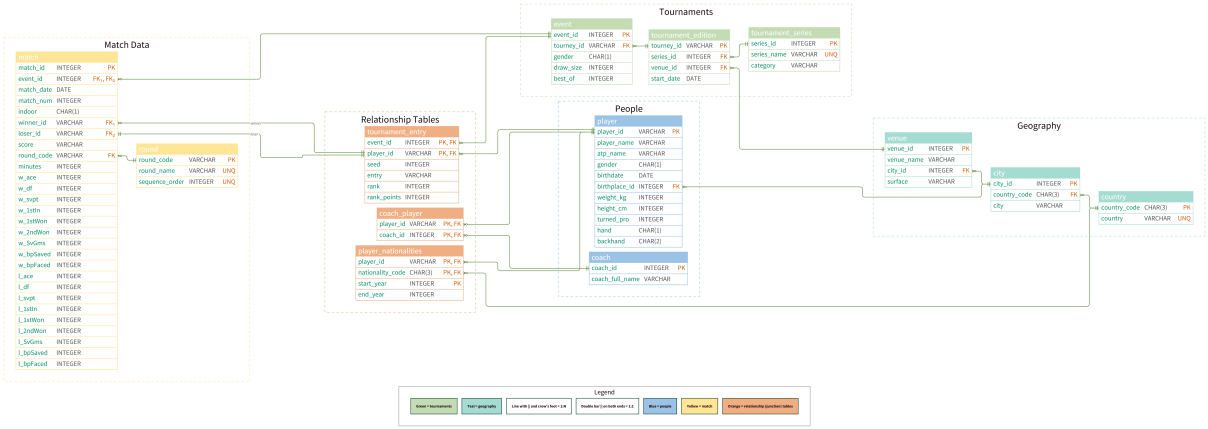
Therefore, with Rule 1 we created 10 **entity** tables (country, city, venue, player, coach, tournament_series, tournament_edition, event, round,match) and with Rule 2 we created 3 **relationship** tables (player_nationalities, coach_player, tournament_entry).

To make these 13 tables easier to read and to represent in the ER diagram, we grouped them under 5 groups based on the kind of information they hold. These groups do not have any effect on the schema.

- **Geography:** tables that contain information related to locations. They are separated by level so that each piece of information is stored once and can be reached through foreign keys.
- **People:** tables that contain information about the people involved in the tennis tournaments.
- **Tournaments:** tables that contain information about the tournaments themselves. They are separated based on how often the information changes. Series stay the same every year, an edition of a series belongs to a specific year, and an event is about a draw within a specific edition.
- **Match data:** tables that contain information about what happens on the match.
- **Relationships:** tables that connect entities from the other groups with many-to-many relationships, and if needed they store attributes describing the connections/relationships.

Group	Table	Type	Table Description
Geography	country	entity	A table that stores countries. Each record is about a single country.
Geography	city	entity	A table that stores cities within countries.
Geography	venue	entity	A table that stores venues around the world where tennis tournaments can take place.
People	player	entity	A table that stores information about individual tennis players.
People	coach	entity	A table that stores information about individual tennis coaches.
Tournaments	tournament_series	entity	A table that stores information about a recurring tournament independent of the year.
Tournaments	tournament_edition	entity	A table that contains information about an edition of a tournament series.
Tournaments	event	entity	A table that contains information about a singles draw (men's or women's) within an edition.
Match data	match	entity	A table that contains information about each singles match played in an event.
Match data	round	entity	A table that stores the stages of a draw match along with its playing order.
Relationships	player_nationalities	relationship (M:N)	A table that contains information about the players and the countries they represent throughout the years.
Relationships	coach_player	relationship (M:N)	A table that contains information about the players and their coaches.
Relationships	tournament_entry	relationship (M:N)	A table that contains information about each player's participation in an event/draw.

These tables give the following **ER diagram**:



Colour legend in the diagram: green = tournaments, teal = geography, blue = people, yellow = match, orange = junction tables. Line with double bar || and crow's foot = 1:N, lines with double bar || on both ends = 1:1.

1.2 Cardinalities and how we represent them

The relationships between our tables are represented in three ways:

- **One-to-many (1:N):** these relationships are represented by a foreign key in the "many" table side pointing to the primary key on the "one" table side.
- **One-to-one (1:1):** this is for a specific case in our tables in which a row of the referenced table is pointed by at most one row of the referencing table.
- **Many-to-many (M:N):** these relationships are represented by a junction table with two foreign keys to the tables that this junction table combines. So, basically an M:N relationship is two 1:N relationships. These relationships are not represented with a single foreign key as this would require to store values in lists which is something not efficient, reproducible and consistent.

A relationship is mandatory on the "many" table side when its foreign key is NOT NULL, and a relationship is optional on the "many" table side when its foreign key can be NULL.

The table below shows every relationship in our database's tables. The cardinality is written from the referenced table to the referencing table.

Relationship	Cardinality	Representation	Explanation
country **contains** city	1:N	FK `city.country_code` → `country.country_code`	A country has many cities, but each city is located in exactly one country.
city **hosts** venue	1:N	FK `venue.city_id` → `city.city_id`	A city can have many different venues, but a venue is located in exactly one city.
player **born in** city	1:N (optional)	FK `player.birthplace_id` → `city.city_id`	Many players can be born in the same city, but each player has at most one birthplace. Birthplace can be NULL if it is unknown.
tournament_series **has** tournament_edition	1:N	FK `tournament_edition.series_id` → `tournament_series.series_id`	A tournament series recurs every year, so a tournament series has many editions, but each edition belongs to exactly one series.
tournament_edition **has** event	1:N	FK `event.tourney_id` → `tournament_edition.tourney_id`	A tournament edition has at most one draw per gender. Each draw belongs to exactly one tournament edition.
venue **hosts** tournament_edition	1:N	FK `tournament_edition.venue_id` → `venue.venue_id`	A venue can host different tournament editions throughout the years, but a tournament edition can be hosted at exactly one venue.
event **has** match	1:N	FK `match.event_id` → `event.event_id`	A draw includes many matches, but each match is played in exactly one draw.
match **is in** round	1:N	FK `match.round_code` → `round.round_code`	Many matches are played in the same round, but each match belongs to exactly one round.
coach **coaches** player	M:N	Junction table `coach_player`. FK `coach_player.player_id` → `player.player_id` & FK `coach_player.coach_id` → `coach.coach_id`	A player can have different coaches, and a coach can coach different players.

Relationship	Cardinality	Representation	Explanation
player **represents** country	M:N	Junction table `player_nationalities`. FK `player_nationalities.player_id` → `player.player_id` & FK `player_nationalities.nationality_code` → `country.country_code`	A player can represent different countries throughout the years, and a country can be represented by different players throughout the years.
player **enters** event	M:N	Junction table `tournament_entry`. FK `tournament_entry.event_id` → `event.event_id` & FK `tournament_entry.player_id` → `player.player_id`	A player can enter different draws throughout the years, and a draw includes different players.
tournament_entry **won** match	1:N	Composite FK `match(event_id, winner_id)` → `tournament_entry(event_id, player_id)`	Each match in a specific draw has one winner. So, a winner tournament entry appears only once in a match, but there can be different matches in the same draw which share the same winner tournament entry.
tournament_entry **lost** match	1:1	Composite FK `match(event_id, loser_id)` → `tournament_entry(event_id, player_id)` with UNIQUE(event_id, loser_id) in match table	Each match has one loser. So, a loser tournament entry appears only once in a match. If a player loses in a match of a specific draw then they are eliminated from that draw. So, there cannot be different matches in the same draw with the same loser tournament entry.

1.3 Primary keys, foreign keys and referential integrity

Primary keys

The table below lists all primary keys for each table in the database along with the additional UNIQUE constraints. The keys that appear in a parenthesis are composite primary keys.

Table	Primary key	Other unique keys
country	country_code	country
city	city_id	(country_code, city)
player	player_id	none
coach	coach_id	none
tournament_series	series_id	series_name
venue	venue_id	(venue_name, city_id)
tournament_edition	tourney_id	(series_id, start_date)
event	event_id	(tourney_id, gender)
round	round_code	round_name, sequence_order
match	match_id	(event_id, match_num), (event_id, loser_id), (event_id, winner_id, round_code), (event_id, winner_id, loser_id)
player_nationalities	(player_id, nationality_code, start_year)	none
coach_player	(player_id, coach_id)	none
tournament_entry	(event_id, player_id)	(event_id, seed)

Foreign keys and policies

Below are clarifications on the policies.

Policy	Meaning
RESTRICT	Immediately blocks deletion or key change if child rows reference the parent.
CASCADE	When deleted, delete the referencing child rows too. On a key update, copies the new key into their foreign-key columns.
SET NULL	Keeps the child rows but sets their foreign-key values to `NULL`.

Each foreign key's value should either be NULL or match an existing primary key value as the referential integrity constraint requires. Each FK below is such a referential integrity constraint. The table lists the policies we chose for every FK, what happens to the referencing rows, and sometimes the reason why we chose that policy.

Foreign key	References	ON DELETE	Explanation ON DELETE	ON UPDATE	Explanation ON UPDATE
city.country_code	country.country_code	RESTRICT	A country cannot be deleted while cities reference it.	CASCADE	A changed country code is copied to its associated cities.
venue.city_id	city.city_id	RESTRICT	A city cannot be deleted while venues reference it.	CASCADE	A changed city ID is copied to its associated venues.
player.birthplace_id	city.city_id	SET NULL	Players remain and their birthplace becomes NULL if the city is deleted (if a city can be deleted under the other foreign-key rules like venue).	CASCADE	A changed city ID is copied to players' associated birthplaces.
tournament_edition.series_id	tournament_series.series_id	RESTRICT	A series cannot be deleted while editions reference it.	CASCADE	A changed series ID is copied to its editions.
tournament_edition.venue_id	venue.venue_id	RESTRICT	A venue cannot be deleted while editions reference it.	CASCADE	A changed venue ID is copied to its editions.
event.tourney_id	tournament_edition.tourney_id	CASCADE	Deleting an edition also deletes the events it contains, unless another constraint blocks this deletion.	CASCADE	A changed tourney ID is copied to its events.
player_nationalities.player_id	player.player_id	CASCADE	Deleting a player also removes their nationality records (the nationalities they represented).	CASCADE	A changed player ID is copied to their nationality records.
player_nationalities.nationality_code	country.country_code	RESTRICT	A country cannot be deleted while nationality records reference it.	CASCADE	A changed country code is copied to the nationality records.
coach_player.player_id	player.player_id	CASCADE	Deleting a player also removes their coaching links.	CASCADE	A changed player ID is copied to their coaching link.
coach_player.coach_id	coach.coach_id	CASCADE	Deleting a coach also removes the coaching links.	CASCADE	A changed coach ID is copied to their coaching links.
tournament_entry.event_id	event.event_id	CASCADE	Deleting an event also removes its entries (unless matches block the deletion).	CASCADE	A changed event ID is copied to its entries.
tournament_entry.player_id	player.player_id	RESTRICT	A player cannot be deleted while tournament entries reference them.	CASCADE	A changed player ID is copied to its entries.
match.event_id	event.event_id	RESTRICT	An event cannot be deleted while matches reference it.	CASCADE	A changed event ID is copied to the corresponding matches.
match.round_code	round.round_code	RESTRICT	A round cannot be deleted while matches reference it.	CASCADE	A changed round code is copied to its matches.

Foreign key	References	ON DELETE	Explanation ON DELETE	ON UPDATE	Explanation ON UPDATE
match (event_id, winner_id)	tournament_entry (event_id, player_id)	RESTRICT	An entry cannot be deleted while it is recorded as a match winner.	CASCADE	Changes to the entry's key are copied to winner references.
match (event_id, loser_id)	tournament_entry (event_id, player_id)	RESTRICT	An entry cannot be deleted while it is recorded as a match loser.	CASCADE	Changes to the entry's key are copied to the loser references.

1.4 SQL file

The file `tennis.sql` creates the whole database, populates the database with data, and runs the five queries for Task2. The steps followed in the sql file:

1. Since SQLite does not check foreign keys if we do not switch them on, the file turns on the FK enforcement and drops any existing tables with the same names, so that it can be run repeatedly without errors.
2. It creates the 13 tables from our design with their attributes and constraints.
3. It populates the database with data that originally come from an Excel file (generated with the help of AI and the publicly available files), but because SQLite cannot read Excel files we included all INSERT queries in the sql file provided. Parent tables are populated before their children so that every FK points to a record that already exists. All inserts run inside one transaction in order to have the whole database loaded with data or nothing, since the tables depend on each other and a failure halfway would leave the database partly loaded and inconsistent, while with the single transaction any error rolls everything back and no table has data if an error occurs.
4. It includes the five queries for Task 2 along with a message printed before each result containing the natural language query.

- **Constraints:** We use constraints in order to have a good quality of data in our database. Primary key and UNIQUE constraints prevent duplicates (e.g. two players with the same seed in the same event), foreign keys ensure that the values referenced exist, NOT NULL enforce for necessary information to not be missing, and CHECK limit the allowed values (e.g. the surface of the courts has specific values, a match's duration cannot be negative). Also, because SQLite does not enforce the length of characters in types like CHAR(3), we have extra CHECK for this too. The code has explainable comments.
- **Triggers:** SQLite does not support the usage of the current date inside a CHECK, so some of the rules regarding dates are implemented as triggers (a player must be at least 14 years old, a player cannot become a professional in the future, a player cannot represent a country starting in a start year that is in the future). So, we use triggers to apply these rules both while inserting and updating information. If a value is inserted or updated that does not follow the rules then the trigger raises an error along with a specific message for that error.
- **Indexes:** We added 8 indexes on the FK columns (SQLite already creates indexes to the PK columns and the columns that are part of the UNIQUE constraints) in order to speed up the joins we use the most.

How to run the database

```
.read tennis.sql
```

1.5 BCNF check of the entity tables

A relation (table) R is in Boyce-Codd Normal Form (BCNF) if, for every FD (functional dependency) X → Y in R, either the FD is trivial or X is a superkey of R.

- **Functional Dependency (FD):** X → Y means that whenever two or more records have the same X value then they also have the same Y value. An FD is **trivial** if Y is subset of X, which is satisfied by all relations.
- **Superkey:** A set of attributes that uniquely identifies a row. A candidate key is a minimal superkey: it can no longer identify a row if any attribute of it is removed. PRIMARY KEY and non-null UNIQUE constraints give the candidate keys.

Basically, BCNF is a rule for table design that prevents and solves the problem of repeated information in a single table. Repeated information can lead to three kinds of anomalies:

1. **Update Anomaly:** When a repeated value is changed only in some rows and not in all the rows it appears.
2. **Insertion Anomaly:** When a fact cannot be stored on its own without requiring unrelated information.
3. **Deletion Anomaly:** When deleting a row removes more facts that were only stored in that row.

Method Used

To identify if our tables are in BCNF we followed some specific steps.

1. Identify the candidate keys.
2. Check that they determine any other column in the table.
3. Check if any other column or set of columns could determine another column in the table.
4. For every assumed non-trivial functional dependency X → Y, check that X is a superkey. If this holds for all such dependencies, the relation is in BCNF.

As explained in 1.1, 10 out of our 13 tables are **entities**, and 3 out of the 13 are **relationships**. Below is the table where we check if all ten entity relations are in BCNF. We also include the relationship tables for completeness.

Relation	Relation Type	Candidate keys	Justification	BCNF
country(**country_code**, country)	Entity	country_code, country	Both columns of the relation are candidate keys, so there is no non-candidate key column that determines another column. All determinants are candidate keys.	Yes
city(**city_id**, country_code, city)	Entity	city_id, (country_code, city)	For city, city_id determines all attributes, and (country_code, city) determines city_id under the chosen uniqueness rule. Neither country_code nor city alone identifies the city_id. All determinants are candidate keys.	Yes
player(**player_id**, player_name, atp_name, gender, birthdate, birthplace_id, weight_kg, height_cm, turned_pro, hand, backhand)	Entity	player_id	All columns that are not candidate keys contain information that different players can share. None of these columns determines another. For example, the height does not determine the weight and vice versa, the hand does not determine the backhand, the birthdate does not determine the year a player became professional since each player can become professional at different ages. All determinants are candidate keys.	Yes
coach(coach_id, coach_full_name)	Entity	coach_id	coach_id determines coach_full_name. Coach names may be shared, so coach_full_name does not determine coach_id. All determinants are candidate keys.	Yes
tournament_series(**series_id**, series_name, category)	Entity	series_id, series_name	The only non-candidate key column is category. But many different series share the same category, so category cannot determine the series. All determinants are candidate keys.	Yes
venue(**venue_id**, venue_name, city_id, surface)	Entity	venue_id, (venue_name, city_id)	The only non-candidate key is surface. The same surface can be shared by different venues, so it cannot determine anything. Also, the venue_name and city_id alone cannot determine anything since a venue with the same name can be located in different cities and a city can have different venues. All determinants are candidate keys.	Yes
tournament_edition(**tourney_id**, series_id, venue_id, start_date)	Entity	tourney_id, (series_id, start_date)	A series can be hosted in different venues throughout the years, and a venue can host different series throughout the years. Also, different tournaments can start on the same date. All determinants are candidate keys.	Yes
event(**event_id**, tourney_id, gender, draw_size, best_of)	Entity	event_id, (tourney_id, gender)	Gender alone cannot determine none of the other columns since there are tournaments for both genders, there are tournaments where men play on best of 5 and best of 3, and the draw size is defined each time separately for men and women. Draw size and best of cannot determine anything since there are a lot of tournaments that share the same information. All determinants are candidate keys.	Yes
round(**round_code**, round_name, sequence_order)	Entity	round_code, round_name, sequence_order	All columns of the relation are candidate keys since every round has a distinct code, name and order, so there is no non-candidate key column that determines another column. All determinants are candidate keys.	Yes
match(**match_id**, event_id, match_date, match_num, indoor, winner_id, loser_id, score, round_code, minutes, w_ace, w_df, w_svpt, w_1stIn, w_1stWon, w_2ndWon, w_SvGms, w_bpSaved, w_bpFaced, l_ace, l_df, l_svpt, l_1stIn, l_1stWon, l_2ndWon,	Entity	match_id, (event_id, match_num), (event_id, loser_id), (event_id, winner_id, round_code)	Each match can be identified by different attributes and combinations of attributes. A match has its own match_id, each match has its own number within a draw, a player can lose only one time in a draw, and a player can win only one time on each round.Each of all these keys determine every other column of the match. However, all the other columns cannot determine each other since date, score, duration, and match statistics are information that different matches can share. The combination of event_id, winner_id, loser_id is also unique but it is a	Yes

Relation	Relation Type	Candidate keys	Justification	BCNF
l_SvGms, l_bpSaved, l_bpFaced)			superkey and not a candidate key. All determinants are superkeys.	
player_nationalities(**player_id**, **nationality_code**, **start_year**, end_year)	Relationship	(player_id, nationality_code, start_year)	The composite key (player_id, nationality_code, start_year) determines end_year. end_year does not determine anything else since different representations of countries can share the same end year. All determinants are candidate keys.	Yes
coach_player(**player_id**, **coach_id**)	Relationship	(player_id, coach_id)	The only candidate key is the pair (player_id, coach_id) and there are no other columns in this table. Since different players can have the same coach and different coaches can coach the same player, no other column can determine anything. All determinants are candidate keys.	Yes
tournament_entry(**event_id**, **player_id**, seed, entry, rank, rank_points)	Relationship	(event_id, player_id), (event_id, seed)	A player can enter a specific event at most once and each seeded player in a specific event can be identified since different players in the same event cannot share the same seed. However, no other columns can determine each other (e.g. players in different events can have the same rank with different rank points). All determinants are candidate keys.	Yes

Note (match.indoor): we treat indoor as a property of the match (we thought it can change based on the weather, so for the same event there may be both indoor and outdoor matches), so there is no extra dependency. However, in the data no event has both indoor and outdoor matches.

Task 2: Querying the database

Query 1: join

Natural Language Question: Which tournaments were played on clay during 2025 season?

The names of the tournaments are stored in tournament_series table, but the surface of the courts are stored in venue table. These two tables are not linked directly, so we should go through the tournament_edition table (that holds the tournaments for 2025 season) to get where each tournament was played in 2025 and get only the ones that were played on clay courts.

Relational Algebra:

$$\pi_{series_name}\left(\sigma_{v.surface='Clay'}\left(\begin{array}{l}\rho_{ts}(tournament_series)\bowtie_{te.series_id=ts.series_id}\rho_{te}(tournament_edition)\\ \bowtie_{v.venue_id=te.venue_id}\rho_{v}(venue)\end{array}\right)\right)$$

SQL:

```sql
-- get the tournament names
SELECT series_name FROM tournament_series ts
-- get the tournaments played on 2025 with their venues
JOIN tournament_edition te ON te.series_id = ts.series_id
-- get the venues and their surface info
JOIN venue v ON v.venue_id = te.venue_id
-- keep only the clay courts
WHERE v.surface = 'Clay'
```

Query 2: aggregate function

Natural Language Question: Across all 2025 events in the database, what is the average match duration in each round, and which round lasts longer?

The match table has all matches for all different events/draws of 2025. We need to group all matches based on the round they were played, and get the average duration of all the matches that were played in the same round. We join match table with round table to get the rounds with their names and not just their codes. The first value is the round that lasts longer on average since we order them in descending order.

Relational algebra:

$$\tau_{avg_minutes\ DESC}\left(\gamma_{r.round_name;\ ROUND(AVG(m.minutes),1)\rightarrow avg_minutes}\left(\begin{array}{l}\rho_{m}(\text{"}match\text{"})\bowtie_{r.round_code=m.round_code}\\ \rho_{r}(round)\end{array}\right)\right)$$

SQL:

```sql
-- get round names and their avg duration with one decimal point
SELECT r.round_name, ROUND(AVG(m.minutes), 1) AS avg_minutes
FROM "match" m
-- round holds the full name of the rounds
JOIN round r ON r.round_code = m.round_code
-- get one result per round
GROUP BY r.round_name
-- the longest round on avg appears first
ORDER BY avg_minutes DESC;
```

Query 3: nested query

Natural Language Question: Which players are taller than the average player in the database?

The average height of the players in the database is not stored somewhere. So, we have to compute it first and then make the comparisons. The nested query calculates the average height of the players in the database and then the outer query returns only the players that are above this average height along with their height.

Relational Algebra:

$$\pi_{player_name,\, height_cm}\left(\sigma_{height_cm > \gamma_{AVG(height_cm)}(player)}\left(player\right)\right)$$

SQL:

```sql
-- get the players and their heights
SELECT player_name, height_cm FROM player
-- keep only the players that their height is above the average height
WHERE height_cm > (SELECT AVG(height_cm) FROM player);
```

Query 4: self join

Question: Which pairs of players were born in the same city?

The place where the players were born is a column on the player table. So, in order to compare the birthplaces between different players we need two copies of the player table (self join). The WHERE condition keeps each pair of players only once and avoids pairing a player with themselves. We also join with the city table to get actual city names instead of city IDs.

Relational algebra:

$$\tau_{c.city}\left(\pi_{c.city,\, p1.player_name \to player_1,\, p2.player_name \to player_2}\left(\sigma_{p1.player_id < p2.player_id}\left(\begin{array}{l}\rho_{p1}(player) \bowtie_{p1.birthplace_id = p2.birthplace_id} \\ \rho_{p2}(player) \\ \bowtie_{c.city_id = p1.birthplace_id} \rho_{c}(city)\end{array}\right)\right)\right)$$

SQL:

```sql
-- get pairs of players born in the same city
SELECT c.city, p1.player_name AS player_1, p2.player_name AS player_2
FROM player p1
-- self join with player to get the 2nd copy of player table
JOIN player p2 on p1.birthplace_id = p2.birthplace_id
-- join with city to get the actual city names
JOIN city c on c.city_id = p1.birthplace_id
-- keep each pair once and avoid pairing a player with themselves
WHERE p1.player_id < p2.player_id
-- see the pairs ordered by city
ORDER BY c.city;
```

Query 5: set operation

Question: Which players have no coach recorded in the database?

We have all the players in the player table, and we have all coach-player links in coach_player table. So, we want the players from the list with all the players that do not appear in the list with the coaching links.

Relational algebra:

$$\tau_{p.player_name}\left(\pi_{p.player_name}\left(\sigma_{p.player_id \in \left(\pi_{player_id}(player) - \pi_{player_id}(coach_player)\right)}\left(\rho_{p}(player)\right)\right)\right)$$

SQL:

```sql
-- get all players that do not have recorded coach links
SELECT p.player_name FROM player p
WHERE p.player_id IN (
    -- all players in the database
    SELECT player_id FROM player
    -- without
    EXCEPT
    -- players that have recorded coach/-es
    SELECT player_id FROM coach_player
)
-- get the results ordered by player name
ORDER BY p.player_name;
```

Task 3: Data Extraction in Python

The executed notebook is `task_3.ipynb`. It creates a fresh temporary tennis.db from `tennis.sql`, connects with sqlite3 and enables foreign-key enforcement. This keeps the analysis independent of any existing database.

Note: The SQL file also contains SQLite commands like `.bail` and `.print` which cannot be executed through Python's `sqlite3` library, so we removed all the lines beginning with "." before executing the SQL code.

3.1 Connecting the database and extracting query results

To show how queries can be sent from Python to the SQLite database, we created a cursor and executed a simple SQL query:

```sql
SELECT player_id, player_name
FROM player
LIMIT 5;
```

We used fetchall() to extract the query results back into Python. This query returned the first five players in the player table, confirming that the database connection was working and that SQL results can be extracted through Python.

3.2 Reading a relation into a Pandas dataframe

For the relation containing a string attribute, we chose the *coach* table. This table contains coach_full_name (a string attribute), together with coach_id. We used `pandas.read_sql_query()` to select these two attributes and store the results in a Pandas DataFrame called df_sim:

```sql
SELECT coach_id, coach_full_name
FROM coach
ORDER BY coach_id;
```

This gave us the dataframe containing **249 coaches**, meaning that the cardinality of the relation is **249,** which is relevant for the hashing function in the next part.

3.3 Hashing function and collision rate

We then created the hashing function for the coach_full_name strings. The requirement was that each string is mapped to an index between 0 and m-1, where m = cardinality of the table. The previous part gave us m = 249, meaning that all hash indexes must be between 0 and 248.

The hashing function starts with a hash value of 0 and goes through every character of each coach's name. The `ord()` function is used to convert each character into a value, and these values are added together. `% m` guarantees that the result is always within m-1.

We applied this function to every coach_full_name in df_sim and stored the result in a new column, hash_index. We then counted the number of unique hash indexes in this column using `nunique()`.

```sql
unique_hashes = df_sim["hash_index"].nunique()
```

A collision occurs when two or multiple strings are mapped to the same hash index. We calculated the number of collisions as the difference between the total number of strings and the number of unique hash indexes:

```
collisions = m - unique_hashes
```

For the 249 coach names, the hashing function produced a collision rate of **39.36%**.

To better understand how the hashing function behaves, we also inspected six similar coach names that were deliberately included by us in the coach table. Their hash indexes were:

Coach name	Hash index
Adam Kellner	124
Adam Kelner	16
Adam Keller	14
Kellner Adam	124
Adam Kellnre	124
Adan Kellner	125

Three of the six similar names (Adam Kellner, Kellner Adam, and Adam Kellnre) receive the same hash index of 124. This is because our hashing function only adds the character codes without looking at the order. Therefore, strings containing exactly the same characters in a different order produce the same sum and therefor the same hash index. Our simple approach for hashing results in collisions when used in a database with similar strings values.

The end

```
print("Thank you Konstantinos & Hakim for reading our (very extensive) report")
```